Linear Probing Revisited: Tombstones Mark the Demise of Primary Clustering

Michael A. Bender
Dept. of Computer Science
Stony Brook University
Stony Brook, NY 11794-2424 USA
bender@cs.stonybrook.edu

Bradley C. Kuszmaul
Google, Inc.
Cambridge, MA, 02142 USA
kuszmaul@gmail.com

William Kuszmaul
CSAIL
Massachusetts Institute of Technology
Cambridge, MA 02139 USA
kuszmaul@mit.edu

2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS) | 978-1-6654-2055-6/22/$31.00 ©2022 IEEE | DOI: 10.1109/FOCS52979.2021.00115

***Abstract*—The linear-probing hash table is one of the oldest and most widely used data structures in computer science. However, linear probing famously comes with a major drawback: as soon as the hash table reaches a high memory utilization, elements within the hash table begin to cluster together, causing insertions to become slow. This phenomenon, now known as primary clustering, was first captured by Donald Knuth in 1963; at a load factor of $1 - 1/x$, the expected time per insertion is $\Theta(x^2)$, rather than the more desirable $\Theta(x)$.**

We show that there is more to the story than the classic analysis would seem to suggest. It turns out that small design decisions in how deletions are implemented have dramatic effects on the asymptotic performance of insertions. If these design decisions are made correctly, then even a hash table that is continuously at a load factor $1-\Theta(1/x)$ can achieve average insertion time $\tilde{O}(x)$. A key insight is that the tombstones left behind by deletions cause a surprisingly strong "anti-clustering" effect, and that when insertions and deletions are one-for-one, the anti-clustering effects of deletions actually overpower the clustering effects of insertions.

We also present a new variant of linear probing, which we call graveyard hashing, that completely eliminates primary clustering on any sequence of operations. If, when an operation is performed, the current load factor is $1 - 1/x$ for some x, then the expected cost of the operation is $O(x)$. One corollary is that, in the external-memory model with a data block size of B, graveyard hashing offers the following remarkable guarantee: at any load factor $1 - 1/x$ satisfying $x = o(B)$, graveyard hashing achieves $1 + o(1)$ expected block transfers per operation. Past external-memory hash tables have only been able to offer a $1 + o(1)$ guarantee when the block size B is at least $\Omega(x^2)$.

Our results come with actionable lessons for both theoreticians and practitioners, in particular, that well-designed use of tombstones can completely change the asymptotic landscape of how the linear probing behaves (and if there are no deletions).

***Keywords*-linear probing; primary clustering; tombstones; graveyard hashing; amortized analysis**

I. Introduction

The linear probing hash table [1, 2, 3] is among the most fundamental data structures to computer science. The hash table takes the form of an array of some size n, where each ***slot*** of the array either contains an element or is empty (i.e., is a ***free slot***). To insert a new element u, the data structure computes a hash $h(u) \in [n]$, and places u into the first free slot out of the sequence $h(u),\ h(u)+1,\ h(u)+2,\ \ldots$ (modulo n). Likewise, a query for u simply scans through the slots, terminating when it finds either u or a free slot.

There are two ways to implement deletions: immediate compactions and tombstones. An immediate compaction rearranges the neighborhood of elements around a deletion to make it as though the element was never there [1, 4]. Tombstones, the approach we default to in this paper, replaces the deleted element with a ***tombstone*** [1, 3]. Tombstones interact asymmetrically with queries and insertions: queries treat a tombstone as a value that does not match the query, whereas insertions treat the tombstone as a free slot. To avoid slowdown from an accumulation of tombstones, the table is occasionally rebuilt and the tombstones are cleared out.

The appeal of linear probing. Linear probing was discovered in 1954 by IBM researchers Gene Amdahl, Elaine McGraw, and Arthur Samuel, who developed the hash table design while writing an assembly program for the IBM 701 (see discussion in [2, 1]). The discovery came just a year after fellow IBM researcher Hans Peter Luhn introduced chained hashing (also while working on the IBM 701) [1]. Linear probing shared the simplicity of chaining, while improving space utilization by avoiding the use of pointers.

The key property that makes linear probing appealing, however, is its data locality: each operation only needs to access one localized region of memory. In 1954, this meant that queries could often be completed at the cost of accessing only a single drum track [2, 3]. In modern systems, it means that queries can often be completed in just a single cache miss [5].[1] The result is that, over the decades, even as computer architectures have changed and as the study of hash tables has evolved into one of the richest and most

[1]Moreover, even when multiple cache misses occur, since those misses are on adjacent cache lines, hardware prefetching can mitigate the cost of the subsequent misses. Both the Masstree [6] and the Abseil B-tree [7] treat the effective cache-line size as 256 bytes even though the hardware cache lines are 64 bytes.

2575-8454/21/$31.00 ©2021 IEEE
DOI 10.1109/FOCS52979.2021.00115

Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:25:35 UTC from IEEE Xplore. Restrictions apply.

studied areas of algorithms, linear probing has persisted as one of the best performing hash tables in practice [5].

The drawback: primary clustering. Unfortunately, the data locality of linear probing comes with a major drawback known as ***primary clustering*** [2, 8]. Consider the setting in which one fills a linear-probing hash table up to a ***load factor*** of $1 - 1/x$ (i.e., there are $(1 - 1/x)n$ elements) and then performs one more insertion. Intuitively, one might expect the insertion to take time $\Theta(x)$, since one in every x slots are free. As Knuth [2] discovered in 1963[2], however, the insertion actually takes expected time $\Theta(x^2)$.

The reason for these slow insertions is that elements in the hash table have a tendency to cluster together into long runs; this is known as primary clustering. Primary clustering is often described as a "winner keeps winning" phenomenon, in which the longer a run gets, the more likely it is to accrue additional elements; see e.g., [9, 10, 11, 12, 13, 14, 15, 16]. As Knuth discusses in [1], however, winner-keeps-winning is not the main cause of primary clustering.[3] The true culprit is the globbing together of runs: a single insertion can connect together two already long runs into a new run that is substantially longer.

Interestingly, primary clustering is an asymmetric phenomenon, affecting insertions but not queries. Knuth showed that if a query is performed on a random element in the table, then the expected time taken is $\Theta(x)$ [2]. This can be made true for all queries (including negative queries) by implementing a simple additional optimization: rather than placing elements at the end of a run on each insertion, order the elements within each run by their hashes. This technique, known as ***ordered linear probing*** [17, 18], allows a query to terminate as soon as it reaches an element whose hash is sufficiently large.[4] Whereas insertions must traverse the entire run, queries need not.

Knuth's result predates the widespread use of asymptotic notation in algorithm analysis. Thus Knuth derives not only the asymptotic formula of $\Theta(x^2)$ for expected insertion time, but also a formula of

$$\frac{1}{2}\left(1 + x^2\right), \tag{1}$$

which is exact up to low order terms. There has also been a great deal of follow-up work determining detailed tail bounds and other generalizations of Knuth's result; see e.g., [19, 20, 8, 21, 22, 23, 24].

The practical and cultural effects of primary clustering. Primary clustering is taught extensively in both theoretical and practical courses [25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36]. Many textbooks not only teach primary clustering [37, 38, 11, 12, 39, 40, 41, 10, 9, 42, 43, 14], but also teach the full formula (1) for insertion time[37, 38, 11, 12, 39, 40, 41, 10, 14].[5]

Because (1) is exact, it is viewed as representing the full picture of how linear probing behaves at high load factors. One consequence is that there has been little empirical work on analyzing the asymptotics of real-world linear probing at high load factors. (And for good reason, what would be the point of verifying what is already known?) As Sedgewick observed in 1990 [38], it is not even clear that the classic formula has been empirically verified at high load factors.

A common recommendation [39, 10, 9, 44, 25, 26, 29, 30, 31, 35] is that, in order to avoid primary clustering, one should use ***quadratic probing*** [45, 46, 47] instead. Whereas linear probing places each element x in the first available position out of the sequence $h(x), h(x) + 1, h(x) + 2, h(x) + 3, \ldots$, quadratic probing uses the first available position out of a quadratic sequence such as $h(x), h(x) + 1, h(x) + 4, h(x) + 9, \ldots, h(x) + k^2, \ldots$ or $h(x), h(x)+1, h(x)+3, \ldots, h(x)+k(k-1)/2, \ldots$. By using a more spread out sequence of probes, quadratic probing seems to eliminate primary clustering in practice. In doing so, quadratic probing also compromises the most attractive trait of linear probing, its data locality. This tradeoff is typically viewed as unfortunate but necessary.

The dangers of primary clustering (and the advice of using quadratic probing as a solution) have been taught to generations of computer scientists over roughly six decades. The folklore advice has shaped some of the most widely used hash tables in production, including the high-performance hash tables authored by both Google [7] and Facebook [48].[6] The consequence is that primary clustering—along with the design compromises made to avoid it—has a first-order impact on the performance of hash tables used by millions of users every day.

This paper. In this paper, we reveal that primary clustering is not the fixed and universal phenomenon that it is reputed to be. When implementing linear probing, there is a small set of design decisions that are typically treated as implementation-level engineering choices. We show that these decisions actually have a remarkable effect on performance: even if a workload operates continuously at a load factor of $1 - \Theta(1/x)$, if the design decisions

[2]Knuth's result [2] was never formally published, and was subsequently rediscovered by Konheim and Weiss in 1966 [8].

[3]This can be easily seen by the following thought experiment. Consider the length ℓ of the run at a given position in the hash table, and then consider a sequence of $(1 - 1/x)n$ insertions where we model each insertion as having probability $(1+\ell)/n$ of increasing the length of the run by 1 (here ℓ is the length of the run at the time of the insertion). The expected length ℓ of the run at the end of the insertions won't be $\Theta(x^2)$. In fact, rather than being an asymptotic function of x, it will simply be $\Theta(1)$.

[4]Contemporary works sometimes also refer to this as ***Robin hood hashing***, but as noted in [18], Robin hood hashing is actually a generalization of ordered linear probing to other open-addressing schemes such as double hashing, uniform probing, etc.

[5]Some books also go through worked examples or tables of (1) to give intuition for how badly linear probing scales, e.g., [38, 12, 41, 14].

[6]Both hash tables are implemented using variants of quadratic probing.

Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:25:35 UTC from IEEE Xplore. Restrictions apply.

are made correctly, then the expected amortized cost per insertion can be decreased all the way to $\tilde{O}(x)$. Our results come with actionable lessons for practitioners, indicating that implementation-level decisions can have unintuitive asymptotic consequences on performance.[7]

We also present a new variant of linear probing, which we call ***graveyard hashing***, that completely eliminates primary clustering on any sequence of operations: if, when an operation is performed, the current load factor is $1 - 1/x$ for some x, then the expected cost of the operation is $O(x)$.

Thus we achieve the data locality of traditional linear probing without any of the disadvantages of primary clustering. One corollary is that, in the external-memory model with a data blocks of size B, graveyard hashing offers the following remarkably strong guarantee: at any load factor $1 - 1/x$ satisfying $x = o(B)$, graveyard hashing achieves $1+o(1)$ expected block transfers per operation. Past external-memory hash tables have only been able to offer a $1 + o(1)$ guarantee when the block size B is at least $\Omega(x^2)$ [50].

What the classical analysis misses. Classically, the analysis of linear probing considers the costs of insertions in an insertion-only workload. Of course, the fact that the final insertion takes expected time $\Theta(x^2)$ doesn't mean that all of the insertions do; most of the insertions are performed at much lower load factors, and the average cost is only $\Theta(x)$.

The more pressing concern is what happens for workloads that operate continuously at high load factors, for example, the workload in which a user first fills the table to a load factor of $1-1/x$, and then alternates between insertions and deletions indefinitely. Now almost all of the insertions are performed at a high load factor. Conventional wisdom has it that these insertions must all suffer from primary clustering.

This conventional wisdom misses an important point, however, which is that the tombstones created by deletions actually substantially change the combinatorial structure of the hash table. Whereas insertions add elements at the ends of runs, deletions tend to place tombstones in the middles of runs. If implemented correctly, the anti-clustering effects of deletions actually outpace the clustering effects of insertions.

We call this new phenomenon ***primary anti-clustering***. The effect is so powerful that, as we shall see, it is even worthwhile to simulate deletions in insertion-only workloads by prophylactically adding tombstones.

Our results flip the narrative surrounding deletions in hash tables: whereas past work on analyzing tombstones [4, 49] has focused on showing that tombstones do not *degrade* performance in various open-addressing-based hash tables, we argue that tombstones actually *help* performance, and are the key to eliminating primary clustering.

[7]The right thing to do may even be the *opposite* of what the literature recommends [4, 49].

A. Results

We begin by giving nearly tight bounds on the amortized performance of ordered linear probing.

There are two design decisions that affect performance: (1) the use of tombstones (which we assume by default) and (2) the frequency with which the hash table is rebuilt and the tombstones are cleared out. Thus, in addition to the size parameter n and the load-factor parameter x, our analysis uses a ***rebuild window size*** parameter R, which is the number of insertions that occur between rebuilds.

The parameter R is classically set to be $n/(2x)$, since this means that the number of tombstone cannot affect the asymptotic load factor. Each rebuild can be implemented in time $\Theta(n)$, so the rebuilds only contribute amortized $\Theta(x)$ time per insertion, which is a low-order term.

A subquadratic analysis of linear-probing insertions. Our first result considers the classical setting $R = n/(2x)$ and analyzes a ***hovering workload***, i.e., an alternating sequence of inserts/deletes at a load factor of $1 - 1/x$.

We prove that the expected amortized cost of each insertion is $\tilde{O}(x^{1.5})$. This is tight, which we establish with a lower bound of $\Omega(x^{1.5}\sqrt{\lg \lg x})$. A surprising takeaway is that the answer is both $\tilde{O}(x^{1.5})$ and $\omega(x^{1.5})$.

This first result is already substantially faster than the classical $\Theta(x^2)$ bound, but it still has several weaknesses. The first (and most obvious) weakness is that we are still not achieving the ideal bound of $\tilde{O}(x)$. The second weakness is that, although the result applies to hovering workloads, it doesn't generalize to *arbitrary* workloads, as can be seen with the following pathological example: consider a workload in which every rebuild window consists of $R-1$ insertions followed by R deletions followed by 1 insertion. The first $R-1$ insertions in each rebuild window cannot benefit at all from tombstones and thus necessarily incur $\Theta(x^2)$ expected time each.

It turns out that both of these weaknesses can be removed if we simply use a larger rebuild window size R. Intuitively, the larger the R, the more time there is for tombstones to accumulate and the better the insertions perform. On the other hand, tombstone accumulation is *precisely* the reason that R is classically set to be small, since it can potentially hurt the performance of queries.

We show that the sweet spot is to set $R = n/\operatorname{polylog}(x)$. Here, the expected amortized cost per insertion drops all the way to $\tilde{O}(x)$, while queries continue to take expected time $O(x)$. Once again, and somewhat surprisingly, the low-order factors are an artifact of reality rather than merely the analysis: no matter the value of R used, either the average insertion cost or the average query cost must be $\omega(x)$.

The bound of $\tilde{\Theta}(x)$ holds not only for hovering workloads, but also for *any workload* that maintains a load factor of at most $1 - 1/x$. Note that here we are analyzing a table in which the capacity n is fixed, and the load factor

Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:25:35 UTC from IEEE Xplore. Restrictions apply.

is permitted to vary over time. It's interesting to see about how the pathological case described above is avoided here. Because the rebuild window is so large, the only way there can be a long series of insertions without deletions is if most of them are performed at low load factors, meaning that they are not slow after all.

A surprising lesson: linear probing is already fast. The core lesson of our results is that linear probing is far less affected by primary clustering than the classical analysis would seem to suggest. Although the classic $\Theta(x^2)$ bound is mathematically correct, it does not capture the amortized cost of insertions at high load factors. This suggests that conclusions that are taught in courses and textbooks, namely that linear probing scales poorly to high load factors, and that alternatives with less data locality such as quadratic probing should be used in its place, stem in part from an incomplete understanding of linear probing and warrant revisiting.

The second lesson is that small implementation decisions can substantially change performance. From a software engineering perspective, our results suggest two simple optimizations (the use of tombstones and the use of large rebuild windows) that should be considered in any implementation of linear probing.

Interestingly, tombstones (and even relatively large rebuild windows) are already present in some hash tables. Thus, one interpretation of primary anti-clustering is as a phenomenon that, to some degree, already occurs around us, but that until now has gone apparently unnoticed.

Graveyard hashing makes linear probing even faster. Our final result is a new version of linear probing, called ***graveyard hashing***, that fully eliminates primary clustering on any sequence of operations. The insight is that, by artificially inserting extra tombstones (not created by deletions), we can ensure that every insertion has good expected behavior.

Insertions, deletions, and queries are performed in exactly the same way as for standard ordered linear probing. The difference is in how we implement rebuilds. In addition to cleaning out tombstones, rebuilds now also insert $\Theta(n/x)$ *new tombstones* evenly spread across the key space.

Graveyard hashing adapts dynamically to the current load factor of the table, performing a rebuild every time that x changes by a constant factor. The running time of each operation is a function of whatever the load factor is at the time of the operation. If a query/insert/delete occurs at a load factor of $1 - 1/x$, then it takes expected time $O(x)$ (even in insertion-only workloads). Graveyard hashing can also be implemented to resize dynamically, so that it is always at some target load factor of $1 - \Theta(1/x)$.

We remark that there is an important sense in which our results on ordered linear probing are orthogonal to our results on graveyard hashing. One set of results revisits the question of how classical versions of linear probing behave, while the other answers the question as to whether it is possible to design new versions of linear probing that do even better.

Coming full circle: improved external-memory hashing. As we mentioned at the outset, one of the big advantages of linear probing is its data locality.

Data locality is formalized via the external-memory model, first introduced by Aggarwal and Vitter in 1988 [51]: a two-level memory hierarchy is comprised of a small, fast ***internal memory*** and a large, slow ***external memory***; blocks can be read and modified only when they are in internal memory, and the act of copying a block from external memory into internal memory is referred to as a ***block transfer***. The model has two parameters, the number B of of records that fit into each block and the number M of records that fit in internal memory. Performance is measured by the number of block transfers that an algorithm or data structure incurs. This model can be used to capture an algorithm's I/O performance (internal memory is RAM, external memory is disk, and block transfers are I/Os) or cache performance (internal memory is cache, external memory is RAM, and block transfers are cache misses).

Ideally, a hash table incurs only amortized expected $1+o(1)$ block transfers per operation, even when supporting a high load factor $1-1/x$.[8] We call the problem of achieving these guarantees the ***space-efficient external-memory hashing problem***. Standard linear probing is a solution when the block size B is very large ($B = \omega(x^2)$), but otherwise, due to primary clustering, it is not [54].

For $B \neq \omega(x^2)$, the state of the art for space-efficient external-memory hashing is due to Jensen and Pagh [50]. They give an elegant construction showing that, if the block size B is $\Theta(x^2)$, then it is possible to achieve amortized expected $1 + O(1/x)$ block transfers per operation, while maintaining a load factor of $1 - O(1/x)$. (In contrast, standard linear probing requires $B = \Omega(x^3)$ to achieve the same $1 + O(1/x)$ result.) However, if $B = o(x^2)$, then no solutions to the problem are known.

Graveyard hashing enables linear probing to be used directly as a solution to the space-efficient external-memory hashing problem, matching Jensen and Pagh's bound for $B = \Theta(x^2)$, and offering an analogous guarantee for arbitrary block sizes $B > \omega(x)$. If $B = \Theta(xk)$ for some $k > 1$, then the amortized expected cost of each operation is $1+O(1/k)$ block transfers. This means that, even if the block size B is only *slightly* larger than the load factor parameter x, we still get $1 + o(1)$ block transfers per operation.

Additionally, graveyard hashing is cache oblivious [55, 54], meaning that the block size B need not be known by the data structure. Consequently, if a system has a multi-level

[8]Many hash tables that are otherwise very appealing perform poorly on this front. For example, if one uses cuckoo hashing, then negative queries require ≥ 2 block transfers, and insertions at high load factor require $\omega(1)$ block transfers [52, 53].

Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:25:35 UTC from IEEE Xplore. Restrictions apply.

hierarchy of caches, each of which may have a different set of parameters B and M, then the guarantee above applies to every level of cache hierarchy.

II. Notation and Conventions

Throughout the paper, we use standard interval notation, where $[m]$ means $\{1, 2, \ldots, m\}$, $[i, j]$ means $\{i, i+1, \ldots, j\}$, $(i, j]$ means $\{i+1, i+2, \ldots, j\}$, etc.

When discussing an ordered linear probing hash table, we use n to denote the number of ***slots*** (i.e., ***positions***). We use $1 - 1/x$ to refer to the ***load factor*** (i.e., the fraction of slots that are taken by elements), and R to refer to the ***rebuild-window size*** (i.e., the number of insertions that must occur before a rebuild is performed). We use S to denote the sequence of operations being performed, and we refer S as the ***workload***.

The operations on the hash table make use of a ***hash function*** $h : U \to [n]$, where U is the universe of possible ***keys*** (also known as ***records*** or ***elements***). We shall assume that h is uniform and fully independent, but as we discuss in the extended paper [56], our results also hold for natural families of hash functions such as tabulation hashing (and, for the analysis of graveyard hash tables, also 5-independent hashing). We can also refer to the ***hash*** $h(u)$ of either a tombstone u (i.e., the hash of the element whose deletion created u) or of an operation u (i.e., the hash of the key that the operation is inserting/deleting/querying).

Each slot in the hash table can either contain a key, be empty (i.e., a free slot), or contain a tombstone. Any maximal contiguous sequence of non-empty slots forms a ***run***. With ordered linear probing, the keys/tombstones in each run are always stored in order of their hash.

Our analysis will often discuss ***sub-intervals*** $I = [i, j] \subseteq [n]$ of the slots in the hash table. We say that an element u ***hashes to*** I if $h(u) \in I$ (but this does not necessarily mean that u resides in one of the slots I). We say that an interval I is ***saturated*** if it is a subset of a run.

Formally, operations in an ordered linear probing hash table are implemented as follows. A query for a key u examines positions $h(u), h(u) + 1, \ldots$ until it either finds u (in which case the query returns true), finds an element with hash greater than $h(u)$ (in which case the query returns false), or finds a free slot (in which case the query also returns false). A deletion of a key u simply performs a query to find the key, and then replaces it with a tombstone. Finally, an insertion of a key u examines positions $h(u), h(u) + 1, \ldots$ until it finds the position j where u belongs in the run; it then inserts u into that position, and shifts the elements in positions $j, j+1, j+2, \ldots$ each to the right by one until finding either a tombstone or a free slot. We say that the insertion ***makes use*** of that tombstone/free slot. Finally, rebuilds are performed every R insertions, and a rebuild simply restructures the table to remove all tombstones. Throughout the paper, n is fixed and does not get changed during rebuilds, although when we describe graveyard hashing (Section IV), we also give a version that dynamically resizes n.

There are two ways to handle overflow off the end of the hash table. One option is to wrap around, meaning that we treat 1 as being the position that comes after n; the other is to extend the table by $o(n)$ (i.e., it ends in slot $n+o(n)$), so that operations never fall off the end of the table. Both solutions are compatible with all of our results. For concreteness, we assume that the wrap-around solution is used, but to simplify discussion, we treat the slots that we are analyzing as being sufficiently towards the middle of the table that we can use the $<$-operator to compare slots.

III. Analyzing Ordered Linear Probing with Tombstones

In this section, we give a technical overview of our analysis of ordered linear probing; we defer the full analysis to the extended version of this paper [56].

A. Understanding the classic $\Theta(x^2)$ bound

We begin by describing the intuition behind Knuth's classic bound, which says that if we fill an ordered linear-probing hash table from empty up to a load factor of $1-1/x$, then the final insertion takes expected time $\Theta(x^2)$.

The fact that the insertion takes time $\omega(x)$ can be attributed to primary clustering, but why does the running time end up being $\Theta(x^2)$ specifically? This turns out to be a result of how standard deviations work. Consider an interval I of x^2 slots in the hash table. The expected number of items that hash into I is $(1 - 1/x)\, x^2 = |I| - x$. On the other hand, the standard deviation for the number of such items is $\Theta(x)$. It follows that, with probability $\Omega(1)$, the number of items that hash into I is $\Omega(x)$ greater than $|I|$. If we then consider the interval I' consisting of the x^2 slots that follow I, then this interval I' must handle not only the items that hash into it, but also the overflow elements from I. The result is that, with probability $\Omega(1)$, the interval I' is fully saturated and forms a run of length x^2.

The above argument stops working if we consider intervals of size $\omega(x^2)$, because the standard deviation on the number of items that hash into I stops being large enough to overflow the interval. The result is that runs of length $\Theta(x^2)$ are relatively common, but that longer runs are not. This is why the expected running time of the insertion performed at load factor $1 - 1/x$ is $\Theta(x^2)$.

Now suppose that, after reaching a load factor of $1-1/x$, we perform a query in our ordered linear-probing hash table. Unlike an insertion, which takes expected time $\Theta(x^2)$, the query takes expected time $\Theta(x)$. We can again see this by looking at standard deviations.

If the query hashes to some position j and takes time t, then there must be at least t elements u that have hashes $h(u) \leq j$ but that reside in positions j or larger. Hence, there

Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:25:35 UTC from IEEE Xplore. Restrictions apply.

is some sub-interval $I = [j_0, j]$ of the hash table (ending in position j) that has overflowed by at least $t-1$ elements, that is, the number of elements that hash to I is at least $|I|+t-1$. As before, the interval that matters most ends up being the one of size $\Theta(x^2)$, and the expected amount by which it overflows is proportional to the standard deviation $\Theta(x)$ of the number of items that hash into the interval.

Thus, the running times of both insertions and queries are consequences of the same two facts: that (a) for any interval I of size x^2, there is probability $\Omega(1)$ that the interval overflows by $\Theta(x)$ elements; and that (b) a given interval of size $\omega(x^2)$ most likely doesn't overflow at all. The only difference is that the running time of an insertion is proportional to the *size* of the interval that overflows (i.e., $\Theta(x^2)$), but the running time of a query is proportional to the *amount* by which the interval overflows (i.e., $\Theta(x)$).

B. The effect of tombstones on a hovering workload

In this subsection, we consider a hovering workload, that is, a sequence of operations that alternates between insertions and deletions on a table with load factor $1-1/x$. We set the size of the rebuild window to be $R = n/(2x)$ (i.e., the value that it is classically set to).

Theorem 1. *Consider an ordered linear probing hash table that uses tombstones for deletions. Suppose that the table is initialized to have capacity n and load factor $1-1/x$, and consider a rebuild window containing a sequence of $R = n/(2x)$ operations that alternate between insertions and deletions.*

The expected amortized time per insertion is $\tilde{O}(x^{1.5})$. If all insertions/deletions are on distinct keys, then the expected amortized time per insertion is $\Omega\left(x^{1.5}\sqrt{\lg \lg x}\right)$. Finally, any queries performed between insertions/deletions take expected time $O(x)$.

We now give a proof sketch for Theorem 1—the full proof can be found in the extended version of this paper [56]. We remark that one of the interesting components of the analysis is its structure: we perform a series of problem transformations, taking us from the question of how to analyze ordered linear probing to a seemingly very different question involving the combinatorics of monotone paths on a grid. By applying geometric arguments to the latter, we end up achieving nearly tight bounds for the former.

Analyzing displacement instead of running time. Define the ***peak*** p_u of an insertion u to be either the hash of the tombstone that the insertion uses (if the insertion makes use of a tombstone) or the position of the free slot that the insertion uses (if the insertion makes use of a free slot). Define the ***displacement*** d_u of an insertion to be $d_u = p_u - h(u)$.

One of the subtleties of how displacement is defined is that, if an insertion u uses a tombstone v, then the displacement measures the difference between $h(u)$ and the *hash* $h(v)$, rather than the difference between $h(u)$ and the *position* of v. This ends up being important for how displacement is used in the analysis[9], but it also means that the displacement of an insertion can potentially be substantially smaller than the running time. For example, if the insertion hashes to position 7 and makes use of a tombstone with hash 13 that resides in position 54, then the displacement is only $13-7=6$ but the running time is proportional to $54-7=47$.

Although we skip the proof here, one can bound the expected difference between displacement and running time by $O(x)$. Thus, even though displacement is not always the same as running time, any bound on average displacement also results in a bound on average running time.

Relating displacement to crossing number. Rather than analyzing the displacement of each *individual* insertion, we bound the average displacement over all R insertions in the rebuild window by relating the displacements to another set of quantities that we call the ***crossing numbers*** $\{c_j\}_{j\in[n]}$. The crossing number c_j counts the number of insertions u in the rebuild window that have a hash $h(u) < j$ but that have a peak $p_u \geq j$ (we consider even the insertions that are subsequently deleted). Each insertion u increments d_u different crossing numbers c_j. Thus

$$\sum_u d_u = \sum_{j\in[n]} c_j.$$

Because we are analyzing the insertions in a rebuild window of size R, the summation on the left side has $R = \Theta(n/x)$ terms, while the summation on the right side has n terms. Thus, if we consider a random insertion u and a random position j, then

$$\mathbb{E}[d_u] = \Theta(x\mathbb{E}[c_j]).$$

If our goal is to establish that the average insertion takes time $o(x^2)$, then it suffices instead to show that the average crossing number c_j is $o(x)$.

Notice that the ratio between $\mathbb{E}[d_u]$ and $\mathbb{E}[c_j]$ is a function of the rebuild window size R; this will come into play later when we consider larger rebuild windows.

Dependencies between insertions/deletions. What makes the analysis of a given crossing number c_j interesting is the way in which insertions and deletions interact over time. If an insertion u has hash $h(u) < j$, and there is a tombstone v with hash $h(v) \in [h(u), j)$, then u can make use of the tombstone and avoid contributing to c_j. But, in order to

[9]The reason for this is actually very simple. Whenever a deletion v is performed, the value of $h(v)$ is fixed (it depends only on the hash function) but the position of v is not (it depends on the other elements in the table, and will change over time). Thus, it is cleaner to measure the deletion's effect on future insertions in terms of $h(v)$ (the thing that is fixed) rather than v's position (the thing that is not).

Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:25:35 UTC from IEEE Xplore. Restrictions apply.

determine whether a given tombstone v is present during the insertion u, we must know whether any past insertions have already used v. That, in turn, depends on which tombstones were present during past insertions, resulting in a chain of dependencies between operations over time.

One of the insights in this paper is that the interactions between insertions and deletions over time can be reinterpreted as an elegant combinatorial problem about paths on a two-dimensional grid. We now give the transformation.

The geometry of crossing numbers. For the sake of analysis, define Z to be the state that our table would be in if we performed only the insertions in the rebuild window and not the deletions.

Since $R = n/(2x)$, the load factor of Z is at most $1 - 1/x + R/n = 1 - 1/(2x)$, which by the classic analysis of linear probing means that the expected distance from any position to the next (and previous) free slot is $O(x^2)$.

Now consider some crossing number c_j. Let j' be the position of the closest free slot to the left of j in Z, that is, the largest $j' < j$ such that position j' is a free slot in Z. Then the only insertions/deletions that we need to consider when analyzing c_j are those that hash into $I = [j' + 1, j)$.

We know from our analysis of Z that $\mathbb{E}[|I|] = O(x^2)$. Although $|I|$ is a random variable with mean $\Theta(x^2)$, to simplify our discussion in this section, we shall treat $|I|$ as simply deterministically equaling x^2. We also treat I as containing no free slots (even at the beginning of the time window). In particular, we know from the classical analysis of linear probing that any interval I of size x^2 has probability $\Omega(1)$ of containing no free slots, so there is no point in our analysis trying to make use of potential free slots in I.

We can visualize the insertions and deletions that hash into I by plotting them in a two-dimensional grid, as in Figure 1a. The vertical axis represents time flowing up from 1 to $2R$, and the horizontal axis represents the hash locations in the interval I. We draw a blue dot in position (i, t) if the t-th operation is an insertion with hash $i \in I$, and we draw a red dot in position (i, t) if the t-th operation is a deletion with hash $i \in I$. (Note that most operations do not hash to I and thus do not result in any dot.)

In order for a given insertion u to be able to make use of a given deletion v's tombstone, it must be that (a) $h(u) \leq h(v)$;[10] that (b) u occurs temporally after v; and that (c) v's tombstone is not used by any other insertion temporally before u. The first two criteria (a) and (b) tell us that for a given insertion (i.e., blue dot) in the grid, the insertion can only make use of tombstones from deletions (i.e., red dots) that are below it and to its right.

Define the set of ***monotone paths*** through the grid to be the set of paths that go from the bottom left to the top right of the grid, and that never travel downward or leftward. Define the ***blue-red deviation*** of such a path to be the number of blue dots below the path minus the number of red dots below the path (see Figure 1a for an example).

What do these monotone paths have to do with the crossing number c_j? Monotone paths with large blue-red deviations serve as witnesses for the crossing number c_j also being large. Suppose that there is a monotone path γ with blue-red deviation $r > 0$. Since we assume that I is initially saturated, each of the insertions (i.e., blue dots) below γ must either use the tombstone left by a deletion (i.e., red dot) that is also below γ or contribute 1 to c_j. Since there are r more blue dots than red dots below γ, we have $c_j \geq r$.

In fact, this relationship goes in both directions (although the other direction requires somewhat more work; see the extended paper [56]). If the crossing number c_j takes some value r, then there must also exist some monotone path with blue-red deviation at least r. The result is that, if we wish to prove either upper or lower bounds on $\mathbb{E}[c_j]$, it suffices to instead prove bounds on the largest blue-red deviation of any monotone path through the grid.

Formalizing the blue-red deviation problem. Let us take a moment to digest the combinatorial problem that we have reached, since on the face of things it is quite different from the problem that we started at.

The expected number of blue dots (and also of red dots) in our grid is $R \cdot |I|/n = \Theta(x)$. If we break the grid into $\sqrt{x}$ rows and $\sqrt{x}$ columns (see Figure 1b for an example), then each cell of the broken-down grid expects to contain $\Theta(1)$ blue and red dots. To simplify our discussion here, think of each cell as independently containing a Poisson random variable $\text{Pois}(1)$ number of blue dots and a Poisson random variable $\text{Pois}(1)$ number of red dots.[11]

Furthermore, rather than considering all monotone paths through the grid, we can restrict ourselves exclusively to the paths that stay on the row and column lines that we have drawn (for an example, see Figure 1b). With high probability in x, this restriction changes the maximum blue-red deviation of any path by at most $\tilde{O}(\sqrt{x})$.

In summary, we have a $\sqrt{x} \times \sqrt{x}$ grid where each cell of the grid contains a Poisson random variable $\text{Pois}(1)$ number of blue points (resp. red points). Whereas our original grid was much taller than it was wide (its height was $2R$ and its width was x^2), our new grid is a $\sqrt{x} \times \sqrt{x}$ square. There are $\binom{2\sqrt{x}}{\sqrt{x}} = \exp(\Omega(x))$ monotone paths γ through the grid, and we wish to prove bounds on the maximum blue-red deviation D achieved by any such path.

Understanding blue-red deviations. To gain intuition, let

[10] Recall that ordered linear probing only ever moves elements to the right over time, meaning that a given insertion u will only use a tombstone if that tombstone has hash at least $h(u)$.

[11] This allows for us to ignore two minor issues in our discussion here: (1) the fact that a single key can potentially be inserted, deleted, and reinserted, resulting in blue and red dots whose horizontal coordinates are deterministically equal; and (2) the fact that the numbers of blue/red dots in each cell are actually very slightly negatively correlated.

Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:25:35 UTC from IEEE Xplore. Restrictions apply.

us start by considering the trivial path γ that contains the entire grid beneath it. The expected number of blue dots (as well as red dots) beneath γ is $\Theta(x)$, and the standard deviation on the number of blue dots (as well as red dots) is thus $\Theta(\sqrt{x})$. With probability $\Omega(1)$, there are $\Omega(\sqrt{x})$ more blue dots in the grid than red dots, which results in a blue-red deviation of $\Omega(\sqrt{x})$ for γ.

Of course, that's just the blue-red deviation of a *single fixed path*. What should we expect the *maximum* blue-red deviation D over all paths to be? On one hand, there are exponentially many paths that we must consider, but on the other hand, the blue-red deviations of the paths are closely correlated to one another. The result, it turns out, is that D ends up being an $x^{o(1)}$ factor larger than $\sqrt{x}$.

We will show that, with probability $1-o(1)$, the maximum blue-red deviation D is between $\Omega(\sqrt{x \lg \lg x})$ and $\tilde{O}(\sqrt{x})$. If we backtrack to our original problem (i.e., we relate the blue-red deviations to the crossing numbers, the crossing numbers to the displacements, and the displacements to the running times), we get that the amortized cost of insertions is between $\Omega(x^{1.5}\sqrt{\lg \lg x})$ and $\tilde{O}(x^{1.5})$.

An upper bound on the maximum blue-red deviation. The first step in bounding D is to prove a general result about decompositions of monotone paths. Let $k = 4\sqrt{x}$ be the perimeter of the grid. We claim that for any monotone path γ through the grid, it is always possible to decompose the area under γ into disjoint rectangles $R_1, R_2, \ldots$ whose perimeters sum to at most $k \lg k$.

Such a decomposition can be constructed recursively as follows: (1) find the point q halfway along the path, and drop a rectangle from q to the bottom-right-most point in the grid; (2) then recursively construct a rectangular decomposition for the portion of the path prior to q, and recursively construct a rectangular decomposition for the portion of the path after q. (See Figure 1c for an example.)

The recursive decomposition is designed so that the sum of the rectangle perimeters in each level of recursion is at most k, meaning that the sum across all levels of recursion is at most $k \lg k$.

The next step in the analysis is to consider the maximum amount that any given rectangle can contribute to the blue-red deviation of a path. If a given rectangle has area a, then the expected number of blue/red dots in the rectangle is $\Theta(a)$, and with high probability in x, the rectangle as a whole has blue-red deviation $O(\sqrt{a} \lg x)$. This, in turn, means that if a rectangle has perimeter p, then with high probability in x, it has blue-red deviation at most $O(p \lg x)$. Finally, since there are only $O(x^2)$ possible rectangles in the entire grid, and this property holds for each of them with probability $1 - 1/\operatorname{poly}(x)$, the property also holds simultaneously for all of them with probability $1 - 1/\operatorname{poly}(x)$.

Putting the pieces together, we know that every path γ has a rectangular decomposition such that the sum of the rectangle perimeters is $O(\sqrt{x} \lg x)$. We further know that, if a rectangle in the decomposition has perimeter p, then it contributes at most $O(p \lg x)$ to the blue-red deviation of γ. It follows that the total blue-red deviation of any path γ is at most $O(\sqrt{x} \lg^2 x)$.

A lower bound on the maximum blue-red deviation. Now we turn our attention to proving a lower bound on D. We wish to find a monotone path γ whose blue-red deviation is $\Omega(x^{1.5}\sqrt{\lg \lg x})$.

Call a $j \times j$ square within the grid ***high-value*** if it has blue-red deviation at least $\Omega(j\sqrt{\lg \lg x})$. The definition is designed so that every square has probability at least $1/\sqrt{\lg x}$ of being high-value.

We construct a monotone path γ recursively as follows. First break the grid into quadrants, and check whether the top left quadrant is a high-value square. If so, then return the trivial path that contains the entire grid below it. Otherwise, recursively construct a path through the bottom-left quadrant, recursively construct a path through the top-right quadrant, and set γ to be the concatenation of the two paths.[12] (For an example, see Figure 1d.)

There are two types of base cases in the recursion. The first type is when a subproblem finds a high-value square; we call this a ***successful base case***. The second type is when a subproblem terminates because it is on a 1×1 grid; we call this a ***failed base case***.

If a successful base case takes place on a sub-grid of width w, then the high-value square that it discovers contributes $\Omega(w\sqrt{\lg \lg x})$ to the total blue-red deviation of γ. In order to establish a lower bound of $\Omega(\sqrt{x \lg \lg x})$ on the blue-red deviation, it therefore suffices to show that the sum of the widths of the successful base cases is $\Omega(\sqrt{x})$.[13]

By construction, the sum of the widths of both the successful base cases and the failed base cases is exactly $\sqrt{x}$. Moreover, each failed base case has width exactly 1. Thus our task reduces to bounding the number of failed base cases by $o(\sqrt{x})$ with probability $1 - o(1)$.

In order for a given failed base case to occur, there is a recursion path of $\Theta(\lg x)$ sub-problems that must all fail to find a high-value square. Each of these failures occurs with probability at most $1 - 1/\sqrt{\lg x}$, so the probability of all of them occurring is

$$\left(1 - 1/\sqrt{\lg x}\right)^{\Theta(\lg x)} = o(1).$$

Since the probability of any given failed base case occurring is $o(1)$, the expected number of failed base cases that occur

[12] When we recurse, we do not change the threshold $\Omega(j\sqrt{\lg \lg x})$ that dictates whether a given $j \times j$ square is special; that is, x acts as a global variable setting this threshold.

[13] There is also a large portion of the area underneath γ that is not contained in any of the high-value squares of the subproblems. Technically, we must also ensure that the blue/red dots in this unaccounted-for area do not substantially change the blue-red deviation, but this follows from a straightforward Chernoff bound.

Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:25:35 UTC from IEEE Xplore. Restrictions apply.

is $o(\sqrt{x})$. By Markov's inequality, the number of failed base cases is $o(\sqrt{x})$ with probability $1 - o(1)$, as desired.

C. Stronger bounds with larger rebuild windows

In this section, we consider what happens if a larger rebuild window size $R = n/\operatorname{polylog}(x)$ is used. As discussed in the introduction, this allows for us to improve our amortized insertion time from $\tilde{\Theta}(x^{1.5})$ to $\tilde{\Theta}(x)$, while still achieving average query time $\Theta(x)$. Moreover, these bounds hold not just for hovering workloads, but also for arbitrary workloads that stay below a load factor of $1 - 1/x$.

Theorem 2. *Let c be a sufficiently large positive constant. Consider an ordered linear probing hash table that uses tombstones for deletions, and that performs rebuilds every $R = n/\lg^c x$ insertions. Finally, consider a sequence of operations S that never brings the load factor above $1-1/x$.*

Then the expected amortized cost of each insertion is $\tilde{O}(x)$ and the expected cost of each query/deletion is $O(x)$.

Moreover, no matter what the rebuild-window size R is, there exists a sequence of operations that takes amortized time $\omega(x)$ per operation.

The full proof of Theorem 2 appears in the extended paper [56]. We focus here on giving a proof sketch of the upper bound. Moreover, to simplify discussion in this section, we continue to focus on the hovering case. Finally, as in the previous section, we shall limit our analysis to a single rebuild window.

There are two main technical challenges that our analysis must overcome. The first challenge is obvious: we must quantify the degree to which tombstones left behind by deletions improve the performance of subsequent insertions. The second challenge is a bit more subtle: in order to support large rebuild-window sizes R, our analysis must be robust to the fact that tombstones can accumulate over time, increasing the effective load factor of the hash table. This latter challenge is further exacerbated by the fact that the choice of which tombstones are in the table at any given moment is a function not only of the sequence of operations being performed, but also of the randomness in the hash table. Thus, even if the cumulative load factor from the elements and tombstones can be bounded (e.g., by $1 - \Theta(1/x)$), we still cannot analyze the tombstones as though they were normal elements; this means we cannot even apply the classic analysis to deduce an $O(x^2)$-time bound for insertions or an $O(x)$-time bound for queries.

Using crossing numbers to rescue the queries. Let us consider the time that it takes to query an element whose hash is j. The time is proportional to the number of elements and tombstones that have hashes smaller than j but that reside in positions j or larger.[14] Call the slots containing these elements ***j-crossed***. Right after a rebuild is performed, the number s of j-crossed slots has expected value $O(x)$. Over the course of the time window between consecutive rebuilds, however, the quantity s gradually increases.

Fortunately, the amount by which s increases is *precisely* the crossing number c_j. Indeed, c_j gets incremented exactly whenever a formerly non-j-crossed slot becomes j-crossed.

Thus, if we can bound c_j to be small then we hit two birds with one stone: we are able to bound the running times of both queries and insertions.

But how do we rescue the crossing numbers? Large rebuild windows also break the analysis of crossing numbers, however. In the original analysis, we argued that there is most likely some position $j' < j$ satisfying $j - j' = O(x^2)$ such that position j' remains an empty slot throughout the entire rebuild window. This meant that, when considering c_j, we only had to analyze insertions and deletions that hash into the interval $I = [j' + 1, j)$ of size $\Theta(x^2)$.

We can no longer argue that such a j' necessarily exists, however, since the accumulation of tombstones over time might eliminate all of the free slots near position j. Thus we must extend our analysis intervals I of size $\omega(x^2)$.

Fortunately, if we consider any interval I of size at least $x^2 \operatorname{polylog} x$, then we can argue that the interval most likely *initially* contains $\Omega(|I|/x)$ free slots. Define the ***insertion surplus*** of an interval I to be the maximum blue-red deviation of any monotone path through the grid representing I, *minus* the number of free slots initially in I. We prove that the crossing number c_j is exactly equal to the maximum insertion surplus of any interval I of the form $[j'+1, j)$. Since large intervals I have a $\Theta(1/x)$-fraction of their slots initially empty, it is very unlikely that they end up determining the crossing number c_j. The result is that we can again focus primarily on intervals of size $O(x^2)$, and perform the analysis of c_j as before.

Putting the pieces together. We now a sequence of $R = n/\operatorname{polylog}(x)$ insertions/deletions. As before, we use the displacement d_u for an insertion as a proxy for insertion time (although bounding the difference between the two requires a more nuanced argument than before, see [56]). We can then relate the displacements to the crossing numbers by

$$\sum_u d_u = \sum_{j \in [n]} c_j.$$

Now, however, both sums consist of between $n/\operatorname{polylog} x$ and n terms. This means that for a random insertion u and a random $j \in [n]$,

$$\mathbb{E}[d_u] \leq \mathbb{E}[c_j \operatorname{polylog}(x)].$$

As before, we can transform the problem of bounding c_j into the problem of bounding the maximum blue-red deviation of any monotone path in a certain grid. The expected number

[14] Technically, we must also consider elements that hash to exactly j but the expected number of such elements is $O(1)$.

Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:25:35 UTC from IEEE Xplore. Restrictions apply.

of blue/red dots in the grid is now $x^2/\operatorname{polylog}(x)$ (rather than $\Theta(x)$). Thus our bound on blue-red deviation comes out as $\tilde{O}(x/\operatorname{polylog}(x)) = O(x)$ (rather than $\tilde{O}(\sqrt{x})$). This means that $\mathbb{E}[c_j] = O(x)$, which implies that the expected time taken by any query is also $O(x)$ and that the average time taken by each insertion is $\tilde{O}(x)$.

We can now also see why $n/\operatorname{polylog}(x)$ is the right rebuild window size to use. In particular, if we make R smaller than $n/\operatorname{poly}(x)$, then insertion times suffer, but if we let R get too close to n (or exceed n) then the crossing numbers c_j become $\omega(x)$ (thanks to our lower bound construction on blue-red deviations), and thus queries take time $\omega(x)$. Thus it is impossible to select a value for R that achieves expected time $O(x)$ for all operations, and if we want $O(x)$-time queries, we must make $R = o(n)$.

IV. Graveyard Hashing

In this section, we describe new variant of linear probing, which we call ***graveyard hashing***. Graveyard hashing takes advantage of the key insight in this paper, which is that tombstones have the ability to significantly improve insertion performance.

Description of graveyard hashing. Graveyard hashing uses different rebuild window sizes, depending on the load factor. If a rebuild is performed at a load factor of $1-1/x$, then the next rebuild will be performed $n/(4x)$ operations later.[15]

Whenever the hash table is rebuilt, Graveyard hashing first removes all of the tombstones that are currently present. It then spreads *new tombstones* uniformly across the table. If the current load factor is $1-1/x$, then $n/(2x)$ tombstones are created, with one tombstone assigned to each of the hashes $\{2ix \mid i \in [n/(2x)]\}$. The purpose of these tombstones is to help all of the up to $n/(4x)$ insertions that occur between the current rebuild and the next rebuild.

The insertion of tombstones during rebuilds is the *only difference* between graveyard hashing and standard ordered linear probing. Thus insertions, queries, and deletions are implemented exactly as in a traditional ordered linear probing hash table.

If desired, one can implement graveyard hashing so that each rebuild also dynamically resizes the table, ensuring that the load factor is always $1 - \Theta(1/x)$ for some fixed parameter x. Note that, when resizing the table, the elements of the table will need to be assigned to new hashes, and thus will need to be permuted. In the RAM model, this can easily be done in linear time (and in place) using an in-place radix sort. In the external-memory model, resizing can be implemented in $O(n/B)$ block transfers (where B is the external memory block size) using Larson's block-transfer efficient scheme for performing partial expansions/contractions on a hash table [57] (this technique has also been used in past work on external-memory hashing, see [50, 54]).

We conclude the section by presenting our two main results on graveyard hashing.

Theorem 3. *Consider a graveyard hash table. For each insertion/query/deletion, if the operation is performed at some load factor of $1-1/x$ then the operation takes expected time $O(x)$, and incurs amortized time $O(x)$ for rebuilds.*

Theorem 4. *Consider the external memory model with $B = rx$ for some $r \geq 1$ and $M = \Omega(B)$. Graveyard hashing can be implemented to offer the following guarantee on any sequence of operations. The load factor of the table is maintained to be $1-1/\Theta(x)$ at all times, and each operation incurs*

$$1 + O(1/r)$$

block transfers in expectation. Furthermore, the amortized block-transfer cost (per operation) of rebuilds is $O(1/r)$.

Corollary 1. *If $x = o(B)$, then the amortized expected number of block transfers per operation is $1 + o(1)$.*

We defer the proofs of these results to the extended version of the paper [56].

Acknowledgment

Michael Bender's work was supported by NSF grants CCF-2106827, CCF-1725543, CSR-1763680, CCF-2118832, CCF-1716252, and CNS-1938709.

William Kuszmaul's work was supported in part by an NSF GRFP fellowship and a Fannie and John Hertz Fellowship. The work was also partially sponsored by the United States Air Force Research Laboratory and was accomplished under Cooperative Agreement Number FA8750-19-2-1000. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the United States Air Force or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein.

References

[1] D. E. Knuth, *The Art of Computer Programming, Volume III: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998. [Online]. Available: https://www.worldcat.org/oclc/312994415

[2] D. Knuth, "Notes on "open" addressing," 1963.

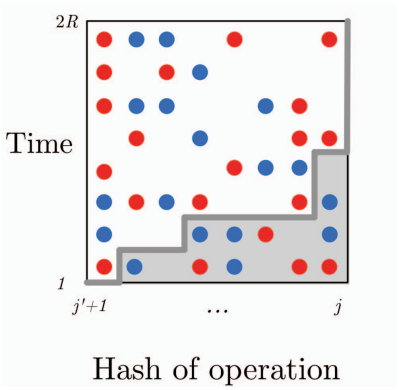
[3] W. W. Peterson, "Addressing for random-access storage," *IBM Journal of Research and Development*, vol. 1, no. 2, pp. 130–146, Apr. 1957.

[4] R. M. Jiménez and C. Martínez, "On deletions in open addressing hashing," in *Proceedings of the Fifteenth Workshop on Analytic Algorithmics and Combinatorics (ANALCO)*. New Orleans, Louisana, USA: SIAM, 8–9 Jan. 2018, pp. 23–31. [Online]. Available: https://doi.org/10.1137/1.9781611975062.3

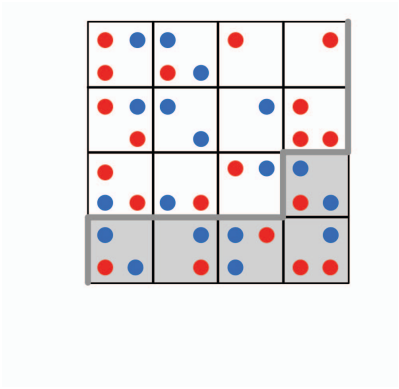
[5] S. Richter, V. Alvarez, and J. Dittrich, "A seven-dimensional analysis of hashing methods and its implications on query processing," *PVLDB – Porceedings of the VLDB Endowment*, vol. 9,

[15] Note that graveyard hashing counts both insertions and deletions as part of the length of a rebuild window.

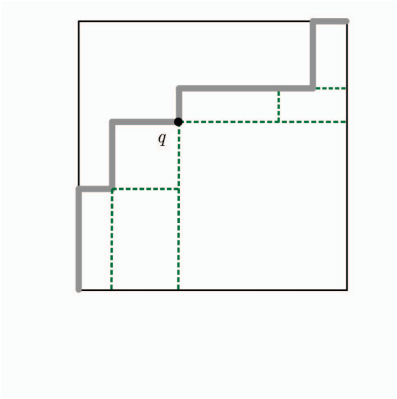
Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:25:35 UTC from IEEE Xplore. Restrictions apply.



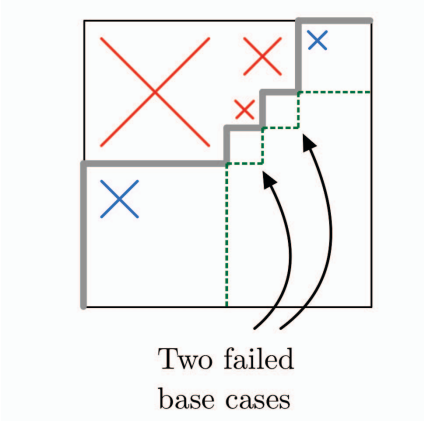
(a) An example of representing insertions/deletions in $I = [j' + 1, j)$ graphically over time. Blue dots are insertions and red dots are deletions. (To simplify this picture, we plot each point in one of eight y-coordinates, but in reality, each dot would have a distinct y-coordinate.) An example monotone path is also given, and the blue-red deviation is $6 - 4 = 2$.



(b) To simplify the problem, we draw a $\sqrt{x} \times \sqrt{x}$ grid, and consider only paths that go along the drawn grid lines. Here $x = 16$.



(c) An example of a recursively constructed rectangular decomposition for a path. The point q used in the top level of recursion is also labeled.



(d) An example of the construction for a path with high blue-red deviation. For each recursive subproblem whose top-left quadrant is not high-value, we place a red X through the quadrant; for each recursive subproblem whose top-left quadrant is high-value, we place a blue X through the quadrant; the only subproblems not to have an X drawn in them are the failed base cases, each of which takes place on a 1×1 grid. There are four base cases, two of which are failed.

Figure 1

no. 3, pp. 96–107, Nov. 2015, the 42nd International Conference on Very Large Data Bases, New Delhi, India. [Online]. Available: https://vldb.org/pvldb/vol9/p96-richter.pdf

[6] Y. Mao, E. Kohler, and R. T. Morris, "Cache craftiness for fast multicore key-value storage," in *Proceedings of the 7th ACM European conference on Computer Systems (EuroSys'12)*, Bern, Switzerland, 10–13 Apr. 2012, pp. 183–196.

[7] "Abseil," 2017, accessed: 2020-11-06. [Online]. Available: https://abseil.io/

[8] A. G. Konheim and B. Weiss, "An occupancy discipline and applications," *SIAM Journal on Applied Mathematics*, vol. 14, no. 6, pp. 1266–1274, Nov. 1966. [Online]. Available: https://doi.org/10.1137/0114101

[9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, Massachusetts, USA: The MIT Press, 2009.

[10] A. Drozdek and D. L. Simon, *Data Structures in C*. Boston, Massachusetts, USA: PWS, 1995.

[11] R. L. Kruse, *Data Structures and Program Design*. Englewood Cliffs, New Jersey, USA: Prentice-Hall Inc, 1984.

[12] H. R. Lewis and L. Denenberg, *Data Structures and Their Algorithms*. New York, New York, USA: HarperCollins Publishers, 1991.

[13] R. Morris, "Scatter storage techniques," *Communications of the ACM*, vol. 11, no. 1, pp. 38–44, Jan. 1968.

[14] J.-P. Tremblay and P. G. Sorenson, *An Introduction to Data Structures with Applications*. McGraw-Hill, 1984.

[15] Wikipedia contributors, "Primary clustering," 2021, accessed 22-May-2021. [Online]. Available: https://en.wikipedia.org/wiki/Primary_clustering

Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:25:35 UTC from IEEE Xplore. Restrictions apply.

[16] P. Smith, *Applied Data Structures with C++*. Jones & Bartlett Learning, 2004.

[17] O. Amble and D. E. Knuth, "Ordered hash tables," *The Computer Journal*, vol. 17, no. 2, pp. 135–142, Jan. 1974.

[18] P. Celis, P. Larson, and J. I. Munro, "Robin Hood hashing (preliminary report)," in *26th Annual Symposium on Foundations of Computer Science (FOCS'85)*, Portland, Oregon, USA, 21–23 Oct. 1985, pp. 281–288. [Online]. Available: https://doi.org/10.1109/SFCS.1985.48

[19] P. Flajolet, P. Poblete, and A. Viola, "On the analysis of linear probing hashing," *Algorithmica*, vol. 22, no. 4, pp. 490–515, Dec. 1998.

[20] A. Viola, "Exact distribution of individual displacements in linear probing hashing," *ACM Transactions on Algorithms (TALG)*, vol. 1, no. 2, pp. 214–242, Oct. 2005.

[21] H. Mendelson and U. Yechiali, "A new approach to the analysis of linear probing schemes," *Journal of the ACM*, vol. 27, no. 3, pp. 474–483, Jul. 1980.

[22] S. Janson and A. Viola, "A unified approach to linear probing hashing with buckets," *Algorithmica*, vol. 75, no. 4, pp. 724–781, Aug. 2016. [Online]. Available: https://doi.org/10.1007/s00453-015-0111-x

[23] A. Viola and P. V. Poblete, "The analysis of linear probing hashing with buckets (extended abstract)," in *Algorithms — ESA '96, Fourth Annual European Symposium*, ser. Lecture Notes in Computer Science, vol. 1136. Springer, 1996, pp. 221–233.

[24] A. Viola, "Distributional analysis of the parking problem and Robin Hood linear probing hashing with buckets," *Discrete Mathematics and Theoretical Computer Science (DMTCS)*, vol. 12, no. 2, Jan. 2010. [Online]. Available: https://dmtcs.episciences.org/519

[25] G. Fisher, "CalPoly CSC103: Fundamentals of computer science – hashing," 2001. [Online]. Available: http://users.csc.calpoly.edu/~gfisher/classes/103/lectures/week5.2.html

[26] G. Kesden, "CMU 15-310: System-level software development — hashing review," 2007, accessed 31-May-2021. [Online]. Available: https://www.andrew.cmu.edu/course/15-310/applications/ln/hashing-review.html

[27] E. D. Demaine and C. E. Leiserson, "MIT 6.046J/18.401J: Introduction to algorithms – Lecture 7: Hashing I," Oct. 2005. [Online]. Available: https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-046j-introduction-to-algorithms-sma-5503-fall-2005/video-lectures/lecture-7-hashing-hash-functions/lec7.pdf

[28] E. Demaine, "MIT 6.897: Advanced data structures – Lecture 10," Spring 2012. [Online]. Available: https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-851-advanced-data-structures-spring-2012/calendar-and-notes/MIT6_851S12_L10.pdf

[29] D. Gries and D. James, "Cornell CS210: Object-oriented programming and data structures — recitation week 8: Hashing," Fall 2014. [Online]. Available: https://www.cs.cornell.edu/courses/cs2110/2014fa/recitations/recitation08/HashPresentation.pptx

[30] D. Bauer, "Columbia COMS W3134: Data structures in Java — Lecture 12: Introduction to hashing," Oct. 2015. [Online]. Available: http://www.cs.columbia.edu/~bauer/cs3134-f15/slides/w3134-1-lecture12.pdf

[31] L. de Greef, "UW CSE 373: Data structures and algorithims — Lecture 7: Hash table collisions," Summer 2017. [Online]. Available: https://courses.cs.washington.edu/courses/cse373/17su/lectures/Lecture%2007%20-%20Hash%20Table%20Collisions.pdf

[32] J. Erickson, "UIUC CS473: Algorithms — Lecture 5: Hash tables," 2017. [Online]. Available: https://jeffe.cs.illinois.edu/teaching/algorithms/notes/05-hashing.pdf

[33] D. Mount, "UMD CMSC 420: Data structures — lecture 11: Hashing — handling collisions," Spring 2019. [Online]. Available: https://www.cs.umd.edu/class/fall2019/cmsc420-0201/Lects/slides-11-hash-collision.pdf

[34] S. Mitra, "UT CS 313E: Elements of software design — hashing," Spring 2021. [Online]. Available: https://www.cs.utexas.edu/~mitra/csSpring2021/cs313/lectures/hash.html

[35] D. G. Sullivan, "Harvard CS S-111: Intensive introduction to computer science using Java — unit 9, part 4: Hash tables," Summer 2021. [Online]. Available: https://sites.fas.harvard.edu/~libs111/files/lectures/unit9-4.pdf

[36] K. Schwarz, "Stanford CS166: Data structures — linear probing," Spring 2021. [Online]. Available: http://web.stanford.edu/class/archive/cs/cs166/cs166.1166/lectures/12/Small12.pdf

[37] R. Sedgewick, *Algorithms*. Reading, Massachusetts, USA: Addison-Wesley, 1983.

[38] ——, *Algorithms in C*. Reading, Massachusetts, USA: Addison-Wesley, 1990.

[39] M. A. Weiss, *Data Structures and Problem Solving using C++*. Reading, Massachusetts, USA: Addison-Wesley, 2000.

[40] M. Main and W. Savitch, *Data Structures and Other Objects Using C++*. Boston, Massachusetts, USA: Addison-Wesley, 2001.

[41] T. A. Standish, *Data Structures, Algorithms, and Software Principles in C*. Reading, Massachusetts, USA: Addision-Wesley, 1995.

[42] M. McMillan, *Data Structures and Algorithms with JavaScript*. Sebastopol, California, USA: O'Reilly, 2014.

[43] M. T. Goodrich and R. Tamassia, *Algorithm Design and Applications*. Hoboken, New Jersey, USA: Wiley, 2015.

[44] Wikipedia contributors, "Linear probing," 2021, accessed 31-May-2021. [Online]. Available: https://en.wikipedia.org/wiki/Lienar_probing

[45] W. D. Maurer, "An improved hash code for scatter storage," *Communications of the ACM*, vol. 11, no. 1, pp. 35–38, 1968. [Online]. Available: https://doi.org/10.1145/362851.362880

[46] F. R. A. Hopgood and J. Davenport, "The quadratic hash method when the table size is a power of 2," *The Computer Journal*, vol. 15, no. 4, pp. 314–315, 1972. [Online]. Available: https://doi.org/10.1093/comjnl/15.4.314

[47] Wikipedia contributors, "Quadratic probing," 2021, accessed 31-May-2021. [Online]. Available: https://en.wikipedia.org/wiki/Quadratic_probing

[48] N. Bronson and X. Shi, "Open-sourcing F14 for faster, more memory-efficient hash tables," 25 Apr. 2019, accessed: 2020-11-06. [Online]. Available: https://engineering.fb.com/2019/04/25/developer-tools/f14/

[49] Attractive Chaos Blog, "Deletion from hash tables without tombstones," Dec. 2019, accessed 22-May-2021. [Online]. Available: https://attractivechaos.wordpress.com/2019/12/28/deletion-from-hash-tables-without-tombstones/

[50] M. S. Jensen and R. Pagh, "Optimality in external memory hashing," *Algorithmica*, vol. 52, no. 3, pp. 403–411, Nov. 2008.

[51] A. Aggarwal and S. Vitter, Jeffrey, "The input/output complexity of sorting and related problems," *Communications of the ACM*, vol. 31, no. 9, pp. 1116–1127, Sep. 1988.

[52] D. Fotakis, R. Pagh, P. Sanders, and P. G. Spirakis, "Space efficient hash tables with worst case constant access time," *Theory of Computing Systems*, vol. 38, no. 2, pp. 229–248, Dec. 2005.

[53] M. Dietzfelbinger and C. Weidling, "Balanced allocation and dictionaries with tightly packed constant size bins," *Theoretical Computer Science*, vol. 380, no. 1-2, pp. 47–68, Jun. 2007.

[54] R. Pagh, Z. Wei, K. Yi, and Q. Zhang, "Cache-oblivious hashing," *Algorithmica*, vol. 69, no. 4, pp. 864–883, 2014. [Online]. Available: https://doi.org/10.1007/s00453-013-9763-6

[55] M. Frigo, C. E. Leiserson, H. Prokop, and S. Ramachandran, "Cache-oblivious algorithms," in *Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS'99)*, 17–19 Oct. 1999, pp. 285–297.

[56] M. A. Bender, B. C. Kuszmaul, and W. Kuszmaul, "Linear probing revisited: Tombstones mark the death of primary clustering," *arXiv preprint arXiv:2107.01250*, 2021.

[57] P.-Å. Larson, "Performance analysis of linear hashing with partial expansions," *ACM Transactions on Database Systems*, vol. 7, no. 4, pp. 566–687, Dec. 1982.

Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:25:35 UTC from IEEE Xplore. Restrictions apply.